

DJAP technical manual

How the audio processing works: every stage, its formulas, measurements and sources

https://github.com/hghaken/DJAP

Free software under the GNU General Public License v3

This manual describes how each part of DJAP's audio processing works: the signal flow, the filters and detectors, the formulas, the reasons behind the design, and how it was verified. It is written for audio engineers, developers and curious users. How to use the program is in the user manual (`DJAP-Manual.pdf`, or `docs/manual.md`); the specification with every decision is `docs/design.md`; the code is in `src/core/` (the DSP, no Windows code) and `src/app/` (the program) - all on https://github.com/hghaken/DJAP.

Numbers in square brackets, such as [1], refer to the sources in chapter 21, with links. All code was written for DJAP; no code or parameter values were taken from other products.

Contents

1. Overview
2. Building blocks
3. Measuring loudness
4. Leveller
5. Colour
6. Compressor
7. Density (multiband)
8. Bass enhancer
9. Exciter
10. De-esser
11. Stereo width
12. Soft clipper
13. Look-ahead true-peak limiter
14. Output, bypass, fades and the Windows volume
15. Meters and graphs
16. The real-time engine
17. Inputs on another clock
18. The plug-in
19. The presets and the standards behind them
20. Verification
21. Sources

1. Overview

1.1 Signal flow per deck

Each deck is one `DeckProcessor` (`src/core/deck.cpp`). Per sample, in this order:

```
input (stereo)
 -> clean-up: non-finite or absurd samples (|x| > 1e6) -> 0; the input factor of the volume follower
(14.4)
 -> [input meter: K-weighted energy, sample peak]           [dry path: the input delayed by the look-
ahead]
 -> leveller gain x input gain                              (4)
 -> colour: sub bass, tone, auto tone, loudness contour      (5)
 -> compressor                                              (6)
 -> density, 1-7 bands                                      (7)
 -> bass enhancer -> exciter -> de-esser                    (8-10)
 -> stereo width                                            (11)
 -> [leveller detector]                                     (4)
 -> soft clipper                                            (12)
 -> look-ahead true-peak limiter: the only delay             (13)
 -> output gain (only down) -> bypass crossfade -> clip guard -> fades
 -> [output meter: K-weighted energy, true peak]
 -> Windows volume (only when followed)                     (14)
output
```

Two decks run the same code independently; with ASIO both are processed in one buffer callback, deck A then deck B, so they share the same latency (they are mixed after DJAP, and a difference would comb-filter).

1.2 Principles

- **Stereo-linked.** Every gain is computed from both channels together and applied to both, so the stereo image never shifts.
- **Double precision.** All filters and gains run in 64-bit floating point; the deck's input and output are 32-bit float. The two channels of a filter run side by side in one SSE2 register (`StereoBiquad`), with the same operations in the same order as the scalar version, so each channel is bit-identical to it.
- **Two rates.** The audio runs per sample; detectors that do not need sample rate (leveller, density, de-esser, auto tone) update every 16 samples (the *control block*, 0.33 ms at 48 kHz), and the gains they produce are ramped linearly across the next block.
- **One delay.** Only the limiter looks ahead (default 1.5 ms, 0..5 ms). No other stage buffers, so the processing's latency is exactly the look-ahead.
- **Nothing clicks.** Every change while playing is ramped or crossfaded (1.4).
- **Bypass is exact.** The bypass path is the input delayed by the same look-ahead, bit for bit.

1.3 Sample rates

44.1, 48, 88.2 and 96 kHz. Every filter is designed for the actual rate from its analogue prototype; the K-weighting of BS.1770, specified at 48 kHz, is re-derived for other rates (2.5).

1.4 Changes without clicks

A new configuration (a preset, a slider, a stage switch) is computed on the GUI thread and handed to the audio thread (16.2). There:

What changes	How it reaches the sound
a stage switched on or off	a 20 ms linear crossfade between the stage's input and output; a stage that starts from rest starts with a cleared state
bypass	a 20 ms crossfade between the processed and the dry path
input and output gain, compressor make-up, width, bass width, bass enhancer amount, clipper drive	a one-pole glide with a 5 ms time constant
filter frequencies of the compressor side chain, exciter and de-esser	new coefficients, filter memory kept
density band count or splits; the width stage's bass split	a second, preallocated filter set is warmed up for 20 ms unheard (a filter started cold on running audio starts with a transient), then the output crossfades to it over 20 ms with a raised cosine (a linear fade's corner was audible on a 120 Hz tone)
bass enhancer frequency	the enhancer fades out (20 ms), takes the new filters, fades back in
colour filters	coefficients ramped per sample from the old design to the new one (5.1)
look-ahead	fade out 5 ms, change the delay, refill it, fade in 5 ms

A configuration that arrives before the first audio block is applied at once, without ramps.

2. Building blocks

2.1 Biquads

Second-order IIR sections in transposed direct form II, double precision, with coefficients from Robert Bristow-Johnson's *Audio EQ Cookbook* [8]: low-pass, high-pass and all-pass (Q = 1/sqrt(2), Butterworth), band-pass and high shelf. When coefficients change the filter keeps its memory (only b0..a2 are copied), so a moving filter does not click. Background on the bilinear transform and digital filters: Smith [9].

2.2 Linkwitz-Riley crossovers

Two identical Butterworth sections in a row form a 4th-order Linkwitz-Riley filter (LR4) [16]. Its low-pass and high-pass outputs at the same frequency are in phase with each other and sum to a 2nd-order all-pass: a flat magnitude. Every band split in DJAP (density, stereo width, bass enhancer) is built from LR4 pairs.

2.3 State-variable filters

The colour stage uses state-variable filters with linear trapezoidal integration, after Andrew Simper [15]. Unlike a biquad, their coefficients may change every sample without disturbing the filter's state, so a gain or a frequency can glide smoothly. The shelves and bells match their analogue prototypes within 0.0001 dB.

2.4 Smoothers

- One-pole: `y = target + a (y - target)`, `a = exp(-1 / (T fs))` - reaches 63 % of a step after T.
- Branching smoother (attack / release): a one-pole with one coefficient when the value grows and another when it falls.
- Linear ramp: at most a fixed step per sample or per block (the leveller's dB/s, the 20 ms switches).

2.5 K-weighting and the loudness scale

ITU-R BS.1770 [1] weights each channel with a high shelf (+4 dB above about 1.5 kHz, the head's effect) and a high-pass (the "RLB" curve, about 38 Hz), then takes the mean square summed over the channels. The loudness is `-0.691 + 10 log10(mean square)` in LUFS. The standard gives the coefficients at 48 kHz; DJAP derives them for any rate through the analogue prototype (re-rated at 44.1 kHz, the response matches the 48 kHz one within 0.005 dB to 16 kHz).

DJAP uses this loudness scale for its detectors too: a compressor threshold "relative to the target" then means a loudness, not a peak level.

2.6 True-peak interpolation

The true peak is the peak of the waveform a converter rebuilds between the samples. It is estimated by oversampling with a polyphase windowed-sinc interpolator (BS.1770 Annex 2 [1]). The prototype filter is centred on a tap, so phase 0 is the sample itself and the other phases lie between samples; each new sample reports the largest magnitude over the interval between two samples. The window is a Kaiser window [10]. Two detectors are used:

Use	Oversampling	Taps per phase	Kaiser beta	Delay
Limiter	8x	96	5	48 frames
Output meter	4x	32	7	16 frames

The limiter's detector is long because a short one misses peaks: 32 taps per phase let full-band noise through 1.07 dB over the ceiling; 96 taps hold it. The inner loops use SSE (four phases per sum, both channels per coefficient load).

2.7 Robustness

- Denormal numbers (very small values in decaying filters) are flushed to zero for each block (FTZ / DAZ) and the processor's previous mode is restored afterwards.
- If any filter or detector state becomes non-finite (one NaN from a broken source would otherwise silence the deck for good), the deck resets its state and counts it.
- The audio thread never allocates, locks, waits, does file I/O or throws (16.1).

3. Measuring loudness

The loudness meter (`src/core/loudness.*`) follows ITU-R BS.1770 [1] and EBU Tech 3341 [3] / 3342 [4]:

- The deck sends a record every 10 ms with the K-weighted energy of input and output.
- **Momentary** loudness: the mean over the last 400 ms. **Short-term**: over 3 s.
- **Integrated**: 400 ms blocks every 100 ms (75 % overlap), gated - blocks under -70 LUFS are ignored (absolute gate), then blocks more than 10 LU under the mean of the rest (relative gate).
- **Loudness range (LRA)**: the spread between the 10th and the 95th percentile of the short-term loudness, after an absolute gate at -70 LUFS and a relative gate 20 LU under the mean [4].

The meter is verified with the EBU Tech 3341 test cases (within 0.1 LU) and LRA cases (within 1 LU) at 44.1 and 48 kHz. The offline renders (`DJAPCmd --render`, `--measure`) use the same meter.

4. Leveller

The leveller is a slow automatic gain that brings each song to the preset's target loudness T. It replaces the per-song gain a playback system would compute in advance from an analysis of the file: a live DJ input has no such analysis, so the leveller has to measure as the music plays.

4.1 Detectors

- **Input level** (0.4 s, K-weighted, before any processing): decides whether the music is *quiet* (below `T - Hold below` after the leveller and input gain), *silent* (below -60 LUFS) or playing.
- **Post level**: measured after the stereo width stage - after compressor and density have done their work - so the leveller aims the *processed* sound at the target. Two averages, 0.4 s (*fast*) and 3 s (*slow*).
- The leveller's own gain of the last block is divided out of the post level. Without this the 3 s average would lag the leveller's own moves and, with a closed loop through it, a fast catch-up overshot to the end of its range.
- The post level is capped at `T + Range + 6 LU`: a single sample far over full scale (and filters ringing after it) is a glitch, not music, and must not pull the gain down.

4.2 The gain

Every control block, while the music plays and is not quiet:

```
wanted = clamp(T - L_slow, -Range, +Range)        (dB; L_slow = the 3 s post level)
gain   = gain moved towards wanted by at most Speed / control rate
```

Range (default 9 dB) is the largest boost or cut; Speed (default 1 dB/s) limits how fast the gain may move, slow enough not to follow the phrases of a song.

4.3 Rules

- **Hold below.** While the input is more than `Hold below` (10 LU) under the target - a fade, a breakdown, a pause - the gain is held. Fades are never pumped up.
- **Fall hold.** While the 0.4 s level is more than 3 LU under the 3 s level (the music is falling), the gain may not rise. This covers the first 10 dB of a fade, before "hold below" applies. Measured: a 40 dB / 10 s fade was lifted 3.2 dB without this rule, 0.65 dB with it.
- **Warm start.** A 3 s average starting from nothing reads new music as far too quiet for seconds. After at least 1 s of quiet, the slow detector follows the fast one for the first 1.5 s of music, and the gain is held meanwhile.
- **Catch-up.** After at least 2 s of silence (a stopped deck) the next song is caught up at 20 dB/s, following the 0.4 s level, so a new song is at level within about half a second. Catch-up goes on while the 0.4 s level still rises more than 1 dB per second (a track fading in), for at most 15 s; then the 3 s detector starts from the level reached. Measured: a 12 s fade-in from -50 dB ended 5.1 LU too loud without, 0.04 LU with.
- **Fast down.** When the gain is more than 3 dB above what the 3 s level needs - a loud song after a quiet intro or a break - it comes down at the *Fast down* speed (default 10 dB/s), following the 0.4 s level (the 3 s average lags a loud entry), only down, until it is within 0.5 dB; then the 3 s level starts from the level reached. Because it is judged on the 3 s level, a chorus 4 LU above the verse does not trigger it. Measured on a 16 dB entry after a 6 s intro: 4-10 s after the entry the output was -6.0 LUFS without fast down and -17.9 LUFS with it (target -18).

4.4 Why the detector sits after the dynamics stages

A leveller that measures before the compressor and density aims the *unprocessed* level at the target; the stages after it then change the loudness again, and the output misses the target by however much they compress or lift. Measuring after them (the handbook's "measure what you control") puts the output on target for every preset.

5. Colour

The colour stage (`src/core/colour.*`) changes the tone of the music and nothing else. It sits after the leveller and before the leveller's detector, so the leveller keeps the loudness on target: a colour changes the balance, not the volume.

5.1 Tone

Four sections, each a state-variable filter (2.3):

Section	Type	Gain	Frequency
Bass	low shelf, Q 0.71	-12..+9 dB	40..250 Hz (100)
Low-mid	bell, Q 0.9	-9..+6 dB	150..1000 Hz (350)
Presence	bell, Q 0.9	-9..+6 dB	1000..6000 Hz (3000)
Treble	high shelf, Q 0.71	-9..+9 dB	3000..16000 Hz (8000)

Targets glide with a 25 ms one-pole. Each section is redesigned once per control block (16 samples) when it moved by at least 0.005 dB or 0.05 Hz, and its coefficients are ramped linearly per sample from the old design to the new one. Stepping them per block instead produced a measurable zipper (a click ratio of 98.9 on a 60 Hz tone; with the ramp below 1). A section that has arrived at 0 dB is skipped and its state cleared, so a neutral colour is bit-exact.

5.2 Auto tone

Pulls each song's spectral balance towards a reference:

1. Five band-passes (Q 1) at 60, 250, 1000, 3000 and 10000 Hz measure the deck's input (mono, after the input gain) and keep a long-term power average per band (time 2..60 s, default 10 s; 1.5 s during the first 6 s of a song so a new song is corrected soon).
2. The *balance* of a band is its level against the 1 kHz band.
3. The correction of the bass, low-mid, presence and treble sections is `clamp((reference - balance) x amount, +-maximum)`, added to the section's own gain. The treble gets half the maximum: the top of an old recording is often its hiss.
4. The averages and the correction hold while the deck is quiet or falling (the leveller's rules), so a breakdown or a fade does not turn the EQ.

The reference is the median balance of a set of late-1980s dance and pop productions: +5.9 / +3.4 / 0 / -1.5 / -5.2 dB at 60 / 250 / 1000 / 3000 / 10000 Hz. `DJAPCmd --tone` prints the balance of any file, so the reference can be recalibrated on other music. Measured corrections at 100 %: a 1956 rock'n'roll single +6 dB bass / +3 dB treble; a late-80s dance track -2.3 / +1.2 dB.

5.3 Sub bass

Adds the octave below the bass, which many records of the 1960s and 1970s lack:

1. The deck's bass below the sub frequency (40..120 Hz) is isolated with a low-pass (two SVF sections).
2. It drives a flip-flop that toggles on every rising zero crossing (with 10 % envelope hysteresis against noise): a square wave at half the frequency - an octave lower.
3. The square wave is multiplied by the bass's envelope and low-passed at 0.6 x the sub frequency (two sections), so only its fundamental remains.

Measured: a 60 Hz tone gets 30 Hz at -2.3 dB (amount 100 %) or -8.3 dB (50 %); silence stays silent. This is the classic octave-divider principle of analogue sub-harmonic synthesisers.

5.4 Loudness contour

At a low listening volume the ear is less sensitive to low and high frequencies (the equal-loudness contours of ISO 226 [17]). The loudness amount adds a low shelf at 100 Hz (up to +9 dB) and a high shelf at 10 kHz (up to +4 dB), measured +8.9 dB at 30 Hz and +3.9 dB at 16 kHz at 100 %.

6. Compressor

A wide-band, feed-forward compressor with a soft knee, after the design analysis of Giannoulis, Massberg and Reiss [6].

1. **Side chain.** The signal after leveller, input gain and colour, through a 2nd-order high-pass (default 80 Hz, 0 = off, up to 200 Hz) so the bass does not trigger the compressor.
2. **Level.** The summed square of both channels, smoothed with a 10 ms one-pole, on the loudness scale: `x = -0.691 + 10 log10(L^2 + R^2)`. A per-sample peak would compress every transient; a short mean square on the loudness scale makes the threshold mean a loudness.
3. **Gain computer** - the static curve in dB, threshold t = T + threshold setting, ratio R, knee width W:

   ```
   x - t < -W/2:          y = x
   |x - t| <= W/2:       y = x + (1/R - 1) (x - t + W/2)^2 / (2W)
   x - t > W/2:           y = t + (x - t) / R
   ```

4. **Smoothing.** The gain reduction `x - y` is smoothed in dB *after* the gain computer, with the attack time when it grows and the release time when it shrinks (the "smooth, decoupled" detector of [6]).
5. **Make-up.** Automatic: the curve's reduction at the target level, so material at the target passes unchanged and only louder material is compressed. Or a fixed value, 0..12 dB.

Measured step response with the defaults: 63 % of the reduction after 14 ms (attack 10 ms plus the 10 ms detector), back to 37 % after 223 ms.

7. Density (multiband)

The density stage compares each frequency band with **its own recent level** rather than with a fixed threshold. Material above that level is turned down, material below it is lifted: soft details come up, peaks are held, and a bass drum moves only the low band. This is what makes the dense, even sound of broadcast processing.

7.1 The band split

A cascade of LR4 splits, each taking its band off the rest of the signal (lowest band first). Every lower band then passes the 2nd-order all-passes of the splits above its own, so all bands share the same phase and sum to an all-pass: a flat magnitude (measured within 0.01 dB for every band count, and 0.001 dB from 30 Hz to 15 kHz with five bands).

```
rest = input
for each split s:  band[s] = LR4 low-pass(rest);  rest = LR4 high-pass(rest)
band[last] = rest
for each band b:   pass band[b] through the all-passes of splits b+1 .. last
```

With K splits this costs K(K-1)/2 all-passes per channel: 7 bands = 6 splits = 12 LR4 filters (24 biquads) and 15 all-passes, 39 biquads per channel. Both channels run in one SSE2 register.

Default splits per band count (each split movable, always rising and at least 1.25x apart):

Bands	Splits (Hz)
1	- (wide-band)
2	200
3	200 / 2500
4	120 / 600 / 4000
5	60 / 200 / 2500 / 6000
6	60 / 200 / 800 / 2500 / 6000
7	60 / 150 / 400 / 1000 / 2500 / 6000

7.2 Per band (both channels together, every control block)

- **Level**: the band's mean square, smoothed over 10 ms.
- **Reference**: the band's level averaged over 3 s - learnt only while the music is not quiet (the leveller's quiet rule, judged on the *incoming* level). It is kept from falling more than 30 dB under the level itself.
- **Excess** = level - reference, in dB. With `k = 1 - 1/ratio` and `ratio = 1 + 3 x Amount` (1:1 at 0 %, 4:1 at 100 %):
  - above `Down above` (3 dB): turned down by `k x (excess - DownAbove)`, at most 24 dB;
  - below `-Lift below` (6 dB): lifted by `k x (-LiftBelow - excess)`, at most `Maximum lift` - never while quiet and never more than 40 dB under the reference.
- The gain moves with the attack time (20 ms) when it goes down and the release time (300 ms) when it goes up, and is ramped linearly across each control block.
- A fixed **band gain** (-6..+6 dB, glided) multiplies on top: with Amount 0 % the stage is a plain multiband tone control.

7.3 Two rules found by testing

1. The quiet rule must look at the *incoming* level, never at the stage's own output: a band that was turned down would otherwise look quiet, stop learning and stay down.
2. The reference must not start empty: an empty reference makes all music look far "above" it and turns it down by up to 12 dB for 1-2 s. For the first 0.5 s of music the reference is learnt fast (0.3 s) and the gain stays at 0 dB.

7.4 Changing the bands while playing

A new band count or new splits change the filters' phase, so they cannot simply be swapped. Each deck holds two complete filter banks (7 bands each, allocated in advance); the new configuration runs in the spare bank, warms up 20 ms unheard and is crossfaded in over 20 ms with a raised cosine. A split change keeps the band levels and gains. Measured cost: density with 7 bands about 0.3-0.4 % of one CPU core per deck.

8. Bass enhancer

Small speakers cannot reproduce the lowest bass, but the ear reconstructs a pitch from its overtones (the "missing fundamental"). Adding overtones of the bass makes it audible on small speakers without adding low-frequency energy (Larsen and Aarts [12], [13]).

1. The mono bass below the enhancer frequency (60..160 Hz, default 100 Hz) is isolated with an LR4 low-pass.
2. It is full-wave rectified, `|x|`: that produces the even overtones 2f, 4f, ...
3. The result is band-passed from f to 4f (2nd-order high-pass and low-pass) and added to both channels with gain `2.4 x Amount` (a rectified sine's 2nd harmonic is 0.42 of the sine; 2.4 brings it to about the same strength). The bass itself is not raised.

Test: 50 Hz gets a 100 Hz overtone 9.4 dB under it at 30 %; off adds nothing; 1 kHz passes unchanged.

9. Exciter

Adds overtones to the highs - shine and air rather than more of the same treble:

1. The highs above the exciter frequency (1500..8000 Hz, default 3500) are isolated with a 4th-order high-pass.
2. They pass a soft saturation `tanh(4x) / 4`. Only **what the saturation adds** is used: `x - tanh(4x)/4` - odd overtones, more on loud highs than on soft ones.
3. That residue is high-passed again (4th order) and mixed in by the amount.

Test: 5 kHz gets 15 kHz 27 dB under it; 500 Hz passes unchanged. Saturation-based enhancement of this kind is described in the DAFX book's chapter on nonlinear processing [7].

10. De-esser

Turns sharp "s", "sh" and "t" sounds down when they stand out:

1. **Detector**: the band above the de-esser frequency (3000..10000 Hz, default 5500) through a 4th-order high-pass, both channels, 2 ms mean square, on the loudness scale.
2. Above the threshold (LU relative to the target, default -10): reduction 4:1 (`0.75 x excess`), at most the maximum (default 6 dB), attack 1 ms, release 80 ms.
3. The reduction is applied as a high shelf at the de-esser frequency (RBJ [8]), recomputed every control block. At 0 dB no filter runs at all (exactly transparent); the shelf's state is cleared when it engages.

Test: a loud 7 kHz tone is turned down 4.5 dB; a 1 kHz tone at the same level is not.

11. Stereo width

Mid / side processing, the principle of Blumlein's 1931 stereo patent [18]:

```
M = (L + R) / 2,   S = (L - R) / 2
S' = BassWidth x LR4low(S) + Width x LR4high(S)       (split at the bass split, 40..400 Hz)
M' = allpass(M)                                        (the same all-pass the LR4 pair has)
L = M' + S',   R = M' - S'
```

- Width 0..200 % (100 % = as recorded, 0 % = mono); bass width 0..100 % (0 % = bass in mono, tight and the same on every speaker; widening the bass is not offered).
- The side is split with an **in-phase LR4 pair** and the mid passes the matching all-pass, so left and right stay in phase with each other. An earlier variant, "side minus its high-pass", is not a clean low band: at 0 % width 4 % of the stereo remained.
- A mono signal stays mono and keeps its level; at 100 / 100 % every frequency keeps its level (only the all-pass phase changes).
- The stage sits before the limiter, because a wider image can mean higher peaks.

Test: a tone on the left channel only (amplitude 1) comes out at 200 % width as L 1.5 and R -0.5, at 0 % as L 0.5 and R 0.5 (mono).

12. Soft clipper

Rounds off the highest peaks without delay, before the limiter, so the limiter has less to do. The signal, scaled to the ceiling (u = 1 is the ceiling) after the drive (0..9 dB), passes a curve that is straight up to half the ceiling and then bends with tanh towards the ceiling - never past it, and without a corner:

```
|u| <= 0.5:   y = u
|u| >  0.5:   y = sign(u) x (0.5 + 0.5 tanh((|u| - 0.5) / 0.5))
```

- The drive makes the music louder and denser by up to its amount; quiet music only gets the gain. It is not compensated (like an input gain).
- **No oversampling**, to keep zero latency. Because the curve has no corner, aliasing stays low: a 3 kHz tone driven 6 dB into the curve gives its strongest alias 42 dB under the tone. First-order antiderivative anti-aliasing was considered and rejected: it acts as a two-sample average and costs up to 4 dB at 15 kHz.
- The meter shows how far the clipper took the deepest peak down.

13. Look-ahead true-peak limiter

The safety net: the output never passes the ceiling (default -1.0 dBTP, -6..0). Look-ahead limiting is described in the DAFX book [7]; DJAP's version (`src/core/limiter.h`) works per sample:

1. **Wanted gain** `w = min(1, ceiling / peak)`, with the peak from the 8x true-peak detector (2.6) - or the sample peak when the look-ahead is shorter than the detector's delay.
2. **Sliding minimum** of w over a window of W samples (a monotonic deque in fixed arrays: O(1) amortised, no allocation).
3. **Moving average** of those minima over the same W samples: the gain reaches its value *before* the peak arrives, along a smooth curve without a step.
4. **Release**: when the average is below the current gain it is taken at once; otherwise the gain rises with a one-pole release (default 80 ms, 10..500 ms).
5. The audio is **delayed** by the look-ahead and multiplied by the gain.

The exact window. The gain applied to the sample leaving the delay line must never be above that sample's own wanted gain. With a sample-peak detector this gives `W = look + 1`; with a true-peak detector of delay D (48 frames here) the wanted gain of a sample is known D frames after it arrives (from the reports of the intervals on both sides of it), so `W = look - D + 1`, and the true peak is used only when `look >= D` (1.0 ms at 48 kHz). The default 1.5 ms is 66 frames at 44.1 kHz and 72 at 48 kHz. A first version was one or two samples off: a single peak went up to 0.4 dB over the ceiling.

Margin. In true-peak mode the limiter aims 0.1 dB under the ceiling: with a 1.5 ms look-ahead its gain changes fast enough to create small inter-sample peaks of its own (full-band noise went 0.17 dB over without the margin).

Guarantee. The sample peak never passes the ceiling. The true peak stays within the ceiling + 0.1 dB for any input, including full-band noise up to the Nyquist frequency, when the look-ahead is at least the detector's delay. Tests: single +12 dBFS samples at look-aheads of 1.5, 0.2, 0.1 and 0 ms; a +6 dBFS sine; an fs/4 square wave (large inter-sample peaks); +9.5 dBFS full-band noise bursts - all measured with a 16x reference true-peak meter.

The moving-average accumulator is re-summed exactly every 8192 samples to stop floating-point drift.

14. Output, bypass, fades and the Windows volume

14.1 Output gain and clip guard

The output gain (-15..0 dB) only turns down, so the ceiling holds. A clip guard then replaces non-finite values with 0 and clamps to [-1, 1]; it never acts on a valid signal, and it is skipped in full bypass so bypass stays bit-exact.

14.2 Bypass

The dry path is the input delayed by the same look-ahead, so processed and dry are aligned in time and the 20 ms bypass crossfade never jumps. Full bypass is the input, bit for bit.

14.3 Fades at start and stop

- Every live start fades the deck's output in over 5 ms, so a stream started in the middle of music does not begin with a step (a hard click). Offline renders and tests start bit-exact without a fade.
- Before a running stream stops - Stop, a new device, Exit, Windows or a setup closing the program - each deck fades to silence over 10 ms (raised cosine) and stays silent; the program waits until every running deck reports silence (at most 300 ms), then 50 ms more so the silence passes the device's buffers.

14.4 Following the Windows volume

Windows applies its volume (keyboard keys, taskbar) only to streams into its default playback device - not to ASIO and not to DJAP's outputs - and if it turned down a virtual cable DJAP reads, the leveller would turn it back up. So DJAP follows that volume itself, when the option is on:

- The program reads the default playback device's volume (in dB, as Windows or the driver applies it, at most 0 dB) and mute 20 times a second.
- Each deck multiplies its output by it after the clip guard (only down, so the ceiling holds; volume 1 is bit-exact), with a 20 ms glide. The meters show the level before it.
- When the default playback device is the deck's own input (a virtual cable) and Windows applies the volume in software, the deck's input is multiplied by the inverse (at most +40 dB), so the processing sees the music's own level and the volume counts once.

15. Meters and graphs

15.1 Meter records

Every 10 ms each deck writes a record into a lock-free ring (16.2): the K-weighted energy of input and output, the sample peak of the input, the true peak of the output (4x detector), the gain of every stage (leveller gain; compressor, de-esser and limiter reduction; density gain per band; how far the clipper reduced the deepest peak), and the energy at 12 points of the chain (`MeterRecord::chain`). The GUI reads the ring 20 times a second.

15.2 Signal-chain bars

For the analysis window the GUI averages the chain energies over 0.4 s and shows, per step, the level after the step against the level before it. The compressor, de-esser, clipper and limiter show their own gain reduction instead, because their level change hides it (make-up gain, the clipper's drive). The bars use a square-root scale so 0.5 dB is visible and 12 dB fills the bar.

15.3 Spectrum

Each deck also copies every input and output frame into a second ring (four floats per frame - the only extra work on the audio thread). On the GUI thread: a 4096-point radix-2 FFT [19] with a Hann window [20], a new frame every 2048 samples, power averaged over 0.3 s, shown at 256 log-spaced points from 20 Hz to 20 kHz (the loudest bin in each point's band, interpolated where the bins are wider than the points). A sine of amplitude A reads 20 log10(A) dB.

15.4 Stereo statistics

Averaged over 0.3 s on the output (and the input for the width):

```
correlation = sum(L R) / sqrt(sum(L^2) sum(R^2))      (+1 mono, 0 unrelated, -1 opposite)
width       = sqrt(sum(S^2) / sum(M^2)) x 100 %       (M = (L+R)/2, S = (L-R)/2; 0 % = mono)
```

The goniometer plots the output's last 43 ms with mono vertical and left / right on the diagonals.

16. The real-time engine

16.1 Rules for the audio thread

The audio callback has a hard deadline (a period of a few milliseconds); missing it is an audible dropout. So the audio thread never allocates memory, takes a blocking lock, does file or network I/O, or throws. All buffers are allocated before the stream starts; the deck's test counts allocations during processing, parameter changes and switches (0).

16.2 Passing data between threads

- **Settings to the audio thread**: the GUI computes a complete coefficient set per deck (`DeckConfig`) and publishes it in a sequence-locked slot (`SeqLockSlot`): the writer increments a counter before and after writing; the audio thread copies the slot and accepts the copy only if the counter was even and unchanged. It never waits.
- **Values from the audio thread**: single-producer / single-consumer rings (`SpscRing`) for the meter records and the analysis frames; a full ring drops data instead of waiting.

16.3 The ASIO buffer switch

With ASIO, the driver calls DJAP once per period with all channels (`AudioEngine::process`): DJAP converts the input buffers from the driver's sample format (all little-endian PCM and float formats), processes deck A then deck B in chunks of at most 256 frames through fixed buffers, converts into the output buffers (clamped, rounded) and signals the driver. Only the channels in use get buffers. Driver reset requests and buffer-size changes are debounced for 500 ms and then handled on the GUI thread by closing and reopening the driver (ASIO SDK [21]).

The other APIs (WASAPI, DirectSound, MME) run through miniaudio [22]: each deck is one duplex stream from its input device to its output device on its own thread.

16.4 Health

Per period the engine measures its own processing time against the period's length: the heaviest and the last period in %, the periods over half their time, the late periods (dropouts) and the periods answered with silence.

16.5 Cost

Both decks with the full chain at 48 kHz use about 3 % of one CPU core (target under 3 %, fail above 5 %). The deck is bound by latency, not by arithmetic: each sample runs one long chain of dependent operations through all stages (a biquad's feedback is about 12 CPU cycles), so faster maths did not help; processing both channels in one SSE2 register and using fewer serial filters did.

17. Inputs on another clock

With ASIO output, a deck's input may come from a WASAPI, DirectSound or MME device - for example a player sending into a virtual cable. The two devices run on their own crystals, so their sample rates differ by tens of ppm (typically up to +-100 ppm); a buffer read at the nominal rate would run dry or overflow within minutes. `AsyncInput` (`src/core/asyncinput.*`) solves this:

- **Ring.** The capture stream writes stereo blocks with a time stamp into a lock-free ring of 32768 frames.
- **Interpolator.** The ASIO side reads it through a 32-tap windowed-sinc interpolator (Kaiser beta 8, cut-off at Nyquist, 256 tabulated fractional positions with linear interpolation between them) - band-limited interpolation as described by Smith [11]. Position 0 is a unit impulse, so at a read speed of exactly 1 the output is the input, bit-exact. Measured: flat within 0.001 dB to 18 kHz, residual below -90 dB to 10 kHz.
- **Drift loop.** The read speed is 1 plus a PI controller on the buffer's fill, at most +-0.2 % (3.5 cents, inaudible). The fill is estimated at the moment of reading (the last block's end plus the time since it came), so the sawtooth of the capture blocks does not modulate the speed; smoothed over 0.5 s, the error corrected over 4 s, the drift estimate following over 16 s (critically damped). With simulated clocks at -500..+500 ppm and late callbacks: drift found within 2 ppm, fill within 0.5 ms of its target, read-speed wobble about 15 ppm, no xruns.
- **Target fill** = the largest capture block + the ASIO period + half the interpolator + a 3 ms margin: about 19 ms for 10 ms capture blocks and 256-frame periods at 48 kHz. Each underrun adds 2 ms of margin (up to 20 ms); the deck fades in over 10 ms whenever reading (re)starts.

This is the only place where DJAP adds buffering beyond the look-ahead, and only when the user chooses such an input. The Setup window shows the added latency and the measured drift.

18. The plug-in

The plug-in (`src/plugin/`, built with Steinberg's plug-in SDK [24]) runs one `DeckProcessor` inside a host such as DJ software. Everything in chapters 2-15 applies unchanged: the plug-in calls the same deck on the host's buffers.

- **Audio.** Stereo in, stereo out, 32-bit float, the host's sample rate and block size (any size, also 1 frame). The host's buffers are processed in place by `DeckProcessor::process`. Activation (outside the audio thread) prepares the deck - the only allocations - and starts the 5 ms fade-in of a live start.
- **One object.** Processor and controller are one object (the SDK's single-component effect), so the plug-in's window reads the deck's meter and tap rings itself through the same `DeckMonitor` (`src/core/monitor.*`) the program uses: one reader per ring, as the rings require (16.2).
- **Parameters.** The 71 values of the chain plus the preset and colour lists, Bypass and the look-ahead. A value's normalised position (0..1) maps to the program's slider positions (`paramPositions`, `paramValueAt`, `paramPositionOf`): linear in the value's steps, logarithmic (300 positions) for times and frequencies spanning a factor of 8 or more. A parameter's id is an FNV-1a hash of its settings key plus 1000, so ids stay the same when values are added or reordered; a test checks that every id is unique.
- **Changes in the audio thread.** Parameter changes arrive with each block; the processor takes the block's last value of each, and when anything changed designs a new `DeckConfig` (`makeDeckConfig`: coefficients only, no allocation) and hands it to the deck through its sequence-locked slot before processing the block. The deck then ramps or crossfades as in 1.4: no automation step clicks.
- **Presets.** Choosing a preset or colour (the list parameters) makes the processor copy that preset's values into its own; the controller loads the same values and asks the host to read all parameters again (`kParamValuesChanged`). The controller keeps the exact values of a preset or a project, so a stored project holds 800 Hz and not the 810 Hz of the nearest log step; a value the host sets is that step's value.
- **Latency.** The look-ahead in samples (`round(ms x rate / 1000)`, 72 at 1.5 ms and 48 kHz) is reported to the host, and announced again (`kLatencyChanged`) when the look-ahead or the sample rate changes, so the host can delay the other channels by the same amount (plug-in delay compensation).
- **State.** A versioned binary stream: preset and colour names, look-ahead, Bypass and every value as a pair of settings key and value. Loading it sets the controller and hands the processor a complete snapshot through a second sequence-locked slot (the host may load a project while audio runs).
- **Window.** A Win32 child window in the host's window, scaled to its DPI, built from the program's own parts: the brushed metal (`theme.*`), the meter panel (`meterview.*`) and tooltips. A DLL must not leave window classes registered after it is unloaded (they would point into freed code), so the classes and GDI+ are registered with the first open window and removed with the last. Common controls version 6 come from the DLL's own manifest (`ISOLATION_AWARE_ENABLED`), whatever the host's manifest says.
- **Verification.** `plugin_test` loads the built plug-in like a host and compares it with a `DeckProcessor` given the same values: bit-exact over 8 s at block sizes 512, 333, 1024, 64 and 1 with a target change half-way; bit-exact after a state is stored and loaded into a new instance; Bypass is the input delayed by the reported latency, bit-exact. The SDK's validator passes all 47 standard and 537 extended tests (bus arrangements, sample rates, block sizes, parameter and state handling, threading rules).

19. The presets and the standards behind them

The presets' targets follow the loudness standards of the media they imitate:

- **EBU R 128** [2] introduced loudness normalisation in broadcasting: programme loudness -23 LUFS, true peak at most -1 dBTP. **EBU Tech 3344** [5] applies it to distribution and reproduction. The **DAB+** preset follows it: -23 LUFS, wide dynamics, songs shifted slowly instead of squashed, compression only above -18 LUFS (R 128's short-term maximum for "s1"), no density, clipping or exciter (a low-bit-rate codec smears clipping and spends its bits on added highs).
- **FM radio** is limited by the multiplex power of ITU-R BS.412 [14] (an average over 60 s), which caps the loudness at about -12..-15 LUFS while station processing makes the sound very dense. The **FM Station** preset processes like a loud station (-9 LUFS) and turns the result down 4 dB, leaving peaks only about 8 dB above the loudness (measured -13.3 LUFS, -5.1 dBTP). It is the sound, not a transmitter-ready signal: no 15 kHz low-pass and no stereo coder.
- **Streaming and file playback**: the AES recommendations for audio streaming and network file playback (TD1004, then TD1008, later the AES77 standard) [23] put music at about -16 LUFS for portable listening; music services normalise to -14..-16 LUFS. The **Web Stream** preset uses -16 LUFS without clipping and with a ceiling of -2 dBTP, because lossy coding raises peaks slightly and turns clipping into audible distortion. **Medium** sits at -14 LUFS.
- The five strength presets (Gentle -18 .. Very Loud -10 LUFS) step about 2 LU at a time; each compresses a little earlier and harder and uses more bands and more density. Gentle sits at the low end of the AES playback range (-16..-20 LUFS); Very Loud approaches modern club masters (-8..-9 LUFS) but keeps a margin so the limiter does not carry everything, and keeps its release at 50 ms or more so a 50 Hz bass is not distorted by the limiter's gain following the waveform.

The reasons for every value are in `docs/design.md` sections 3 and 3.1.

20. Verification

Each area has its own test program (`tests/`, run all with `tools\run_tests.ps1 -Renders`):

Area	Test	Pass
Loudness	EBU Tech 3341 cases and LRA cases (Tech 3342) at 44.1 / 48 kHz; silence	+-0.1 LU; LRA +-1 LU; -inf
K-weighting	44.1 kHz response against 48 kHz	0.005 dB to 16 kHz
True peak	Tech 3341 true-peak cases	+0.2 / -0.4 dB
Compressor	static curve; step response	exact; 63 % in about 14 ms
Limiter	+12 dBFS single samples at several look-aheads; +6 dBFS sine; fs/4 square; +9.5 dBFS noise bursts	sample peak <= ceiling; true peak (16x reference) <= ceiling + 0.1 dB
Bypass	processing off	the input delayed by the look-ahead, bit-exact
Density	band-sum flatness for every band count 1..7	0.01 dB
Band count	3 -> 7 -> 1 bands on a sine while playing	no step larger than the signal's own
Clicks	every parameter changed on a sine in its own band	no step larger than the signal's own
Colour	every section against its analogue prototype; loudness contour; neutral; auto tone; sub bass	0.02 dB; +8.9 / +3.9 dB; bit-exact; reference - balance; 30 Hz -2.3 dB
Width	left-only tone at 200 % and 0 %; mono stays mono	1.5 / 0.5; 0.5 / 0.5
Presets	all 8 presets on test material at -18 and -9 LUFS in	output within 1 LU of each preset's target; true peak <= ceiling
Robustness	NaN / Inf input; FTZ restored	recovers within a block; processor mode unchanged
Real time	allocation counter during processing and changes	0 allocations
Decks	deck A's processing does not change deck B; equal latency	bit-exact; the same delay
CPU	both decks, full chain, 48 kHz	< 3 % of one core (fail > 5 %)
Inputs on another clock	simulated clocks -500..+500 ppm, late callbacks, pauses	no xruns, drift within 3 ppm, fill within 0.5 ms
Graphs	FFT against a DFT; sines 50 Hz - 18 kHz; correlation and width of known signals	1e-12; within a bin; +-0.05 / 1 %
Offline	synthetic songs through every preset, checked with an independent meter	integrated loudness within 1 LU of the target, true peak <= ceiling
Plug-in	loaded like a host: against the deck at block sizes 1..1024 with a change; state stored and loaded; Bypass; latency; parameter ids; the SDK's validator	bit-exact; 71 of 71 values, bit-exact; bit-exact; 72 / 144 samples; unique; 47 + 537 tests passed
Slider positions	every value's positions: range, order, every shipped value back within a step	exact ends, rising, within a step (2 % on log scales)

The offline render check runs the real deck code on synthetic songs (fades, pauses, loud entries) and measures the results with a separate, independently written loudness and true-peak meter.

21. Sources

Standards, papers and books the design is based on. All links were checked on 2026-10-26; some publishers' pages need a membership or a purchase for the full text.

#	Source	Used for
[1]	ITU-R BS.1770-5 (2023), *Algorithms to measure audio programme loudness and true-peak audio level* - https://www.itu.int/rec/R-REC-BS.1770-5-202311-I	K-weighting, gating, true peak (2.5, 2.6, 3)
[2]	EBU R 128, *Loudness normalisation and permitted maximum level of audio signals* - https://tech.ebu.ch/publications/r128	loudness normalisation, LUFS / LU, -1 dBTP (19)
[3]	EBU Tech 3341, *Loudness Metering: "EBU Mode" metering to supplement EBU R 128* - https://tech.ebu.ch/publications/tech3341	momentary / short-term / integrated, test cases (3, 19)
[4]	EBU Tech 3342, *Loudness Range: a measure to supplement EBU R 128* - https://tech.ebu.ch/publications/tech3342	loudness range (3)
[5]	EBU Tech 3344, *Guidelines for distribution and reproduction in accordance with EBU R 128* - https://tech.ebu.ch/publications/tech3344	the DAB+ preset (19)
[6]	D. Giannoulis, M. Massberg, J. D. Reiss, *Digital Dynamic Range Compressor Design - A Tutorial and Analysis*, J. Audio Eng. Soc. 60(6), 2012, pp. 399-408 - https://aes2.org/publications/elibrary-page/?id=16354	compressor structure, soft knee, smoothing (6)
[7]	U. Zoelzer (ed.), *DAFX - Digital Audio Effects*, 2nd ed., Wiley 2011 - https://dafx.de/DAFX_Book_Page_2nd_edition/	look-ahead limiting, nonlinear processing (9, 13)
[8]	R. Bristow-Johnson, *Audio EQ Cookbook* (W3C note) - https://www.w3.org/TR/audio-eq-cookbook/	biquads, shelves (2.1, 10)
[9]	J. O. Smith III, *Introduction to Digital Filters*, CCRMA, Stanford - https://ccrma.stanford.edu/~jos/filters/	bilinear transform, filter structures (2.1)
[10]	J. O. Smith III, *Spectral Audio Signal Processing*: "The Kaiser Window" - https://ccrma.stanford.edu/~jos/sasp/Kaiser_Window.html; after J. F. Kaiser, *Nonrecursive digital filter design using the I0-sinh window function*, Proc. IEEE ISCAS 1974, pp. 20-23	windows of the true-peak and interpolation filters (2.6, 17)
[11]	J. O. Smith III, *Digital Audio Resampling Home Page* - https://ccrma.stanford.edu/~jos/resample/	band-limited interpolation (17)
[12]	E. Larsen, R. M. Aarts, *Reproducing Low-Pitched Signals through Small Loudspeakers*, J. Audio Eng. Soc. 50(3), 2002, pp. 147-164 - https://aes2.org/publications/elibrary-page/?id=11088; summary at https://research.tue.nl/en/publications/reproducing-low-pitched-signals-through-small-loudspeakers/	bass enhancer (8)
[13]	E. Larsen, R. M. Aarts, *Audio Bandwidth Extension: Application of Psychoacoustics, Signal Processing and Loudspeaker Design*, Wiley 2004	bass enhancer (8)
[14]	ITU-R BS.412-9, *Planning standards for terrestrial FM sound broadcasting at VHF* - https://www.itu.int/rec/R-REC-BS.412/en	multiplex power limit, the FM Station preset (19)
[15]	A. Simper, *Linear Trap Optimised SVF* (Cytomic technical paper) - https://cytomic.com/files/dsp/SvfLinearTrapOptimised2.pdf	state-variable filters of the colour stage (2.3, 5)
[16]	S. H. Linkwitz, *Active Crossover Networks for Noncoincident Drivers*, J. Audio Eng. Soc. 24(1), 1976, pp. 2-8 - https://aes2.org/publications/elibrary-page/?id=2649; see also https://www.linkwitzlab.com/filters.htm	Linkwitz-Riley band splits (2.2, 7, 11)
[17]	ISO 226:2023, *Acoustics - Normal equal-loudness-level contours* - https://www.iso.org/standard/83117.html	loudness contour (5.4)
[18]	A. D. Blumlein, British patent 394,325 (filed 1931, granted 1933), *Improvements in and relating to sound-transmission, sound-recording and sound-reproducing systems*	mid / side stereo (11)

#	Source	Used for
[19]	J. W. Cooley, J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Mathematics of Computation 19 (1965) - https://doi.org/10.1090/S0025-5718-1965-0178586-1	FFT (15.3)
[20]	F. J. Harris, *On the use of windows for harmonic analysis with the discrete Fourier transform*, Proc. IEEE 66(1), 1978 - https://doi.org/10.1109/PROC.1978.10837	Hann window (15.3)
[21]	Steinberg, ASIO SDK (used under its GPL-3.0 option) - https://www.steinberg.net/developers/	ASIO driver interface (16.3)
[22]	D. Reid, miniaudio (public domain / MIT-0) - https://miniaud.io/	WASAPI, DirectSound and MME streams (16.3)
[23]	AES Technical Council, TD1004 (2015) and TD1008 (2021), *Recommendations for loudness of internet audio streaming and on-demand distribution* (TD1008 later became the AES77 standard) - AES technical documents: https://aes2.org/?p=50870	streaming targets, the Web Stream preset (19)
[24]	Steinberg Media Technologies, plug-in SDK v3.8.1 (MIT licence; the plug-in interfaces, base classes, validator and test hosts) - https://github.com/steinbergmedia/vst3sdk	the plug-in (18)

Besides these sources the design draws on the project's own measurements, recorded with each decision in `docs/design.md`.